

# Real-world (build and compare the supervisor version) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Both depths below reuse `research()`, `write()`, and `StageResult` from `sequential_measure`'s solution unchanged. Read both depths — they're not "wrong, right," they're 2 real, valid ways to solve the same problem, with real tradeoffs between them.

## Basic Version

### Approach 1 — the direct way, `if`-based routing

```
# architecture_comparison_practice.py – Real-world section
from typing import TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.types import Command

class SupervisorState(TypedDict):
    task: str
    research_text: str
    summary: str
    routing_log: list
    total_tokens: int
    total_seconds: float

def supervisor(state):
    if not state.get("research_text"):
        log = state["routing_log"] + ["research"]
        return Command(update={"routing_log": log}, goto="research_node")
    if not state.get("summary"):
        log = state["routing_log"] + ["write"]
        return Command(update={"routing_log": log}, goto="write_node")
    return Command(goto=END)

def research_node(state):
```

```

    result = research(state["task"])
    return Command(
        update={
            "research_text": result.text,
            "total_tokens": state["total_tokens"] + result.tokens,
            "total_seconds": state["total_seconds"] + result.seconds,
        },
        goto="supervisor",
    )

def write_node(state):
    result = write(state["research_text"])
    return Command(
        update={
            "summary": result.text,
            "total_tokens": state["total_tokens"] + result.tokens,
            "total_seconds": state["total_seconds"] + result.seconds,
        },
        goto="supervisor",
    )

builder = StateGraph(SupervisorState)
builder.add_node("supervisor", supervisor)
builder.add_node("research_node", research_node)
builder.add_node("write_node", write_node)
builder.add_edge(START, "supervisor")
graph = builder.compile()

result = graph.invoke({
    "task": "climate change",
    "research_text": "",
    "summary": "",
    "routing_log": [],
    "total_tokens": 0,
    "total_seconds": 0.0,
})
print(result["routing_log"])
print(result["total_tokens"], "tokens,", result["total_seconds"], "seconds")

```

**Expected output** (exact numbers vary by run):

```

['research', 'write']
306 tokens, 2.81 seconds

```

This works and correctly reuses `sequential_measure`'s functions unchanged. It's missing a fair, run-both comparison against the sequential pipeline, and its routing decision costs nothing — both fine for a first working version.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

## Intermediate Version

### Approach 1 — a `run_supervisor()` report matching `run_sequential()`'s shape

```
# architecture_comparison_practice.py — Real-world section
def run_supervisor(topic: str) -> dict:
    initial_state = {
        "task": topic,
        "research_text": "",
        "summary": "",
        "routing_log": [],
        "total_tokens": 0,
        "total_seconds": 0.0,
    }
    final_state = graph.invoke(initial_state)
    return {
        "routing_log": final_state["routing_log"],
        "total_tokens": final_state["total_tokens"],
        "total_seconds": final_state["total_seconds"],
    }

if __name__ == "__main__":
    topics = ["climate change", "inflation", "sleep quality"]
    for topic in topics:
        sequential_report = run_sequential(topic)    # from sequential_measure
        supervisor_report = run_supervisor(topic)    # this exercise

        seq_tokens = sequential_report["total_tokens"]
        seq_seconds = sequential_report["total_seconds"]
        sup_tokens = supervisor_report["total_tokens"]
        sup_seconds = supervisor_report["total_seconds"]
        sup_log = supervisor_report["routing_log"]

        print(f"\n{topic}")
        print(f"  sequential: {seq_tokens} tokens, {seq_seconds:.2f}s")
        print(f"  supervisor: {sup_tokens} tokens, {sup_seconds:.2f}s")
        print(f"  routed: {sup_log}")
```

**Expected output** (exact numbers vary by run):

```

climate change
  sequential: 306 tokens, 2.75s
  supervisor: 306 tokens, 2.81s
  routed: ['research', 'write']

inflation
  sequential: 286 tokens, 2.60s
  supervisor: 286 tokens, 2.66s
  routed: ['research', 'write']

sleep quality
  sequential: 325 tokens, 2.90s
  supervisor: 325 tokens, 2.96s
  routed: ['research', 'write']

```

**Difference from Basic:** `run_supervisor()` returns the exact same 3 keys ( `total_tokens` , `total_seconds` , plus `routing_log` in place of nothing) as `sequential_measure`'s `run_sequential()` , so the two can be printed side by side without translating field names. Tokens match exactly between sequential and supervisor here — expected, since this `if`-based router adds real wall-clock overhead (the extra node hops) but zero extra tokens, because it never calls the model to decide. That's the exact gap Approach 2 closes.

## Approach 2 — an LLM-based supervisor, so the routing decision's real cost shows up

```

# architecture_comparison_practice.py – Real-world section
import time

def supervisor_llm(state):
    start = time.perf_counter()
    prompt = (
        f"Task: {state['task']}\n"
        f"Research done: {bool(state.get('research_text'))}\n"
        f"Write done: {bool(state.get('summary'))}\n"
        "Reply with exactly one word: RESEARCH, WRITE, or DONE."
    )
    response = model.invoke(prompt)
    elapsed = time.perf_counter() - start
    decision = response.content.strip().upper()
    call_tokens = response.usage_metadata["total_tokens"]

    updates = {
        "total_tokens": state["total_tokens"] + call_tokens,
        "total_seconds": state["total_seconds"] + elapsed,
    }

```

```

if decision == "RESEARCH":
    updates["routing_log"] = state["routing_log"] + ["research"]
    return Command(update=updates, goto="research_node")
if decision == "WRITE":
    updates["routing_log"] = state["routing_log"] + ["write"]
    return Command(update=updates, goto="write_node")
return Command(update=updates, goto=END)

builder_llm = StateGraph(SupervisorState)
builder_llm.add_node("supervisor", supervisor_llm)
builder_llm.add_node("research_node", research_node)
builder_llm.add_node("write_node", write_node)
builder_llm.add_edge(START, "supervisor")
graph_llm = builder_llm.compile()

def run_supervisor_llm(topic: str) -> dict:
    initial_state = {
        "task": topic, "research_text": "", "summary": "",
        "routing_log": [], "total_tokens": 0, "total_seconds": 0.0,
    }
    final_state = graph_llm.invoke(initial_state)
    return {
        "routing_log": final_state["routing_log"],
        "total_tokens": final_state["total_tokens"],
        "total_seconds": final_state["total_seconds"],
    }

```

```

# architecture_comparison_practice.py – Real-world section
if __name__ == "__main__":
    topics = ["climate change", "inflation", "sleep quality"]
    for topic in topics:
        seq = run_sequential(topic)
        sup_if = run_supervisor(topic)
        sup_llm = run_supervisor_llm(topic)

        seq_t, seq_s = seq["total_tokens"], seq["total_seconds"]
        if_t, if_s = sup_if["total_tokens"], sup_if["total_seconds"]
        llm_t, llm_s = sup_llm["total_tokens"], sup_llm["total_seconds"]

        print(f"\n{topic}")
        print(f"  sequential:      {seq_t}t {seq_s:.2f}s")
        print(f"  supervisor (if):  {if_t}t {if_s:.2f}s")

```

```
print(f"  supervisor (llm):  {llm_t}t {llm_s:.2f}s")
print(f"    routed: {sup_llm['routing_log']}")
```

**Expected output** (exact numbers vary by run):

```
climate change
sequential:      306t 2.75s
supervisor (if): 306t 2.81s
supervisor (llm): 341t 3.62s
  routed: ['research', 'write']
```

The LLM-based supervisor calls the model 3 times total (route, route, route-to-done) — roughly 35 extra tokens and 0.8 extra seconds here, entirely from routing decisions that produce zero user-visible work. That's the real, honest cost `paper_design` 's prediction was estimating — an `if` -based router hides it completely.

### Approach 3 — a written comparison table across all 3 designs

```
# architecture_comparison_practice.py – Real-world section
def compare_all(topics: list[str]) -> None:
    header = f"{'topic':<16} {'sequential':>18} {'if':>18} {'llm':>18}"
    print(header)
    for topic in topics:
        seq = run_sequential(topic)
        sup_if = run_supervisor(topic)
        sup_llm = run_supervisor_llm(topic)

        seq_cell = f"{seq['total_tokens']}t/{seq['total_seconds']:.1f}s"
        if_cell = f"{sup_if['total_tokens']}t/{sup_if['total_seconds']:.1f}s"
        llm_cell = f"{sup_llm['total_tokens']}t/{sup_llm['total_seconds']:.1f}s"

        row = f"{topic:<16} {seq_cell:>18} {if_cell:>18} {llm_cell:>18}"
        print(row)

compare_all(["climate change", "inflation", "sleep quality"])
```

**Expected output** (exact numbers vary by run):

| topic          | sequential | if        | llm       |
|----------------|------------|-----------|-----------|
| climate change | 306t/2.8s  | 306t/2.8s | 341t/3.6s |
| inflation      | 286t/2.6s  | 286t/2.7s | 319t/3.4s |
| sleep quality  | 325t/2.9s  | 325t/3.0s | 362t/3.8s |

**Real result, versus the `paper_design` guess:** sequential and the `if`-based supervisor are nearly identical on tokens (routing there was free) and very close on speed (a few extra node hops, no extra model calls). The LLM-based supervisor is the one that actually matches `paper_design`'s prediction — roughly 10-15% more tokens and 25-30% more wall-clock time than sequential, entirely from a routing decision that, for this fixed two-step task, never had a real choice to make.

`paper_design`'s verdict holds: for a task whose order never changes, a supervisor buys nothing except its own overhead.

**Difference between Approach 1 and Approaches 2/3:** Approach 1's comparison only shows sequential against a free-routing supervisor, which understates the pattern's real cost — a production supervisor calls the model to route, it doesn't use `if` checks. Approach 2 adds that missing, honest routing cost by building a second supervisor variant that actually pays for its decision. Approach 3 doesn't add new measurement — it lines up all 3 designs' numbers in one table across multiple topics, which is what turns "the LLM supervisor seemed to cost more" into a specific, repeatable percentage you could put in front of the tough senior engineer this document's "How to Practice" step asks you to defend against.

**Which one should you actually write?** For any real routing decision that could genuinely go more than one way, Approach 2's LLM-based supervisor is the one whose numbers you can trust — an `if`-based router only tells the truth about tasks so simple they didn't need a supervisor in the first place. For a task this fixed, though, the honest conclusion isn't "use the LLM supervisor" — it's "don't use a supervisor at all here," which is exactly what `paper_design` predicted before any code existed. Keep both supervisor variants in your toolkit: `if`-based routing for genuinely fixed pipelines (call it "sequential with extra steps," not really a supervisor), and LLM-based routing only once the next specialist truly can't be known ahead of time — which is what `ambiguous_routing` builds next.